SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. *Introduction:*

Commission Compass tackles the problem at hand by feeding the user's prompt into an MCP server. The MCP server identifies the context of the prompt and classifies each key point identifying the importance of each point. The MCP server then feeds the classification to our GLM model that processes it and comes up with a logical decision formatted for the end user.

1.1. Purpose:

This document serves as a clarification on the design choices behind Commission Compass, its technical scope and implementation.

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    a. *Our app uses the Flutter framework for its Frontend UI as we intend to make this app cross-platform for all users to enjoy. The GLM model itself sits in a Python backend wrapped inside a Docker container for easy testing and collaboration among members.*
2. ***Data Flows:***
    a. *Users first enter a prompt from either their phone/laptop/PC and this travels through our backend routing to the backend which contains the GLM model. The GLM model determines if a prompt requires MCP (Model Context Protocol) tooling (this is because a user could also send a vague prompt like "Hi" which does not require MCP tooling). Then, the data gets sent through MCP's complexity scoring tool for risk assessment, prompt categorising by industry and prompt ambiguity checking. After this, the data flows to MCP's market benchmarking tool to compare the category and scope of commission with standard or average rates globally. The GLM is then fed all information and structures it accordingly. The structured output is then validated on the backend to ensure it follows the strict format imposed before finally sending it back to the user through the Frontend.*
3. ***Model Process:***
    a. *Our model improves the current agentic workflow model by instead having a deterministic one that includes structured reasoning, risk assessment and tool injection. Instead of having the AI determine its own reasoning and recursively loop through thousands of neural networks for a mathematically sound response, our model is special as we use MCP tools to limit the scope to certain aspects that fit our app, only reviewing decisions based on this scope. We utilize tools like MCP complexity scoring in order to avoid false reasonings and to better understand the context of the prompt. The MCP complexity tool allows the AI to assess the current situation and rate it in terms of urgency. For example: A vague, shorter deadline commission will be rated higher than an elaborated, well-timed deadline commission indicating a higher risk. We also use MCP tools to search and compare with market rates in order to better assess the risk of a*

© UMHackathon 2026

decision. Lastly, the model shall compress all of the output into structured format for better user clarity.

4. *Role of Reference:*
    a. *This document is a reference for all the developers and testers to enable the overall high-level architecture of this project.*

1.2. ***Background****:*

Many freelancers often complain to us that they were "tricked" or felt scammed by clients after being asked certain requests that led to difficulties to those freelancers. Additionally, many new freelancers argue that they lack the proper direction to prosper in the "solopreneur" field as there are lack of information online for proper decision making or because they are not aware of how to find such information. These concerns drove us to create such a system.

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Freelancer	*Prompts the AI of a decision-making request*	*Structured response with full clarity*
Development Team	*Build the platform and maintain it*	*Clear API usage contracts, codebase is modular and well-documented architecture*
QA Team	*Validating the system behavior for all the user roles*	*Defined scope, the request and response formats, and coverage of edge cases*

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

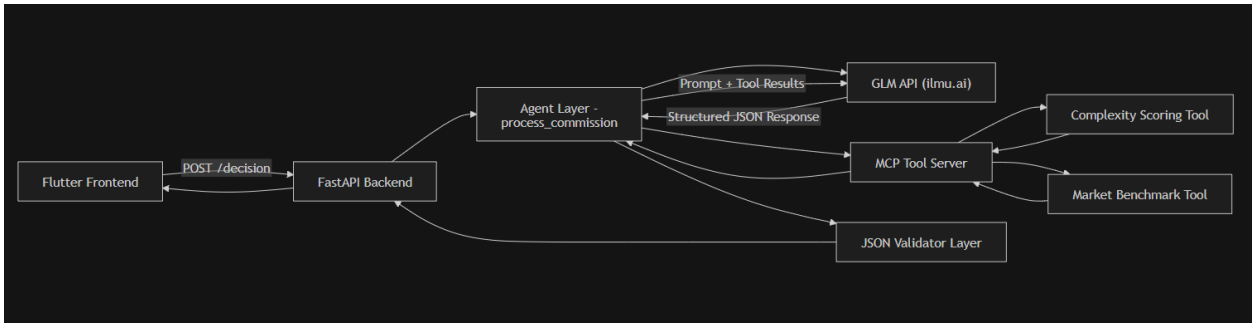
Type	**Details**
System	*Mobile, Web and Desktop App*
Architecture	*Backend server running a Python app*

© UMHackathon 2026

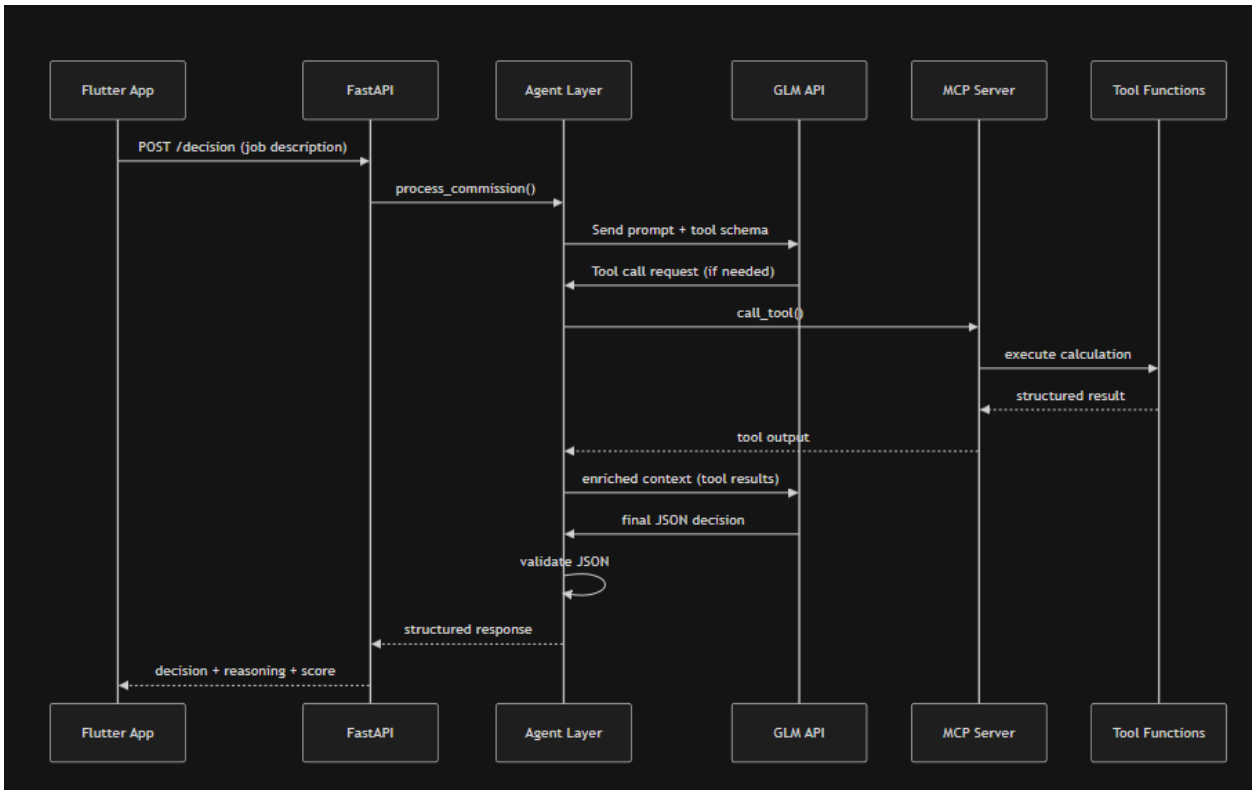
2.1.2 GLM as Decision Service Layer

The GLM acts as our main reasoning engine, tool orchestrator and structured output generator and formattor.

2.1.3 Dependency Diagram



2.2.2. Sequence Diagram



2.2. Technological Stack

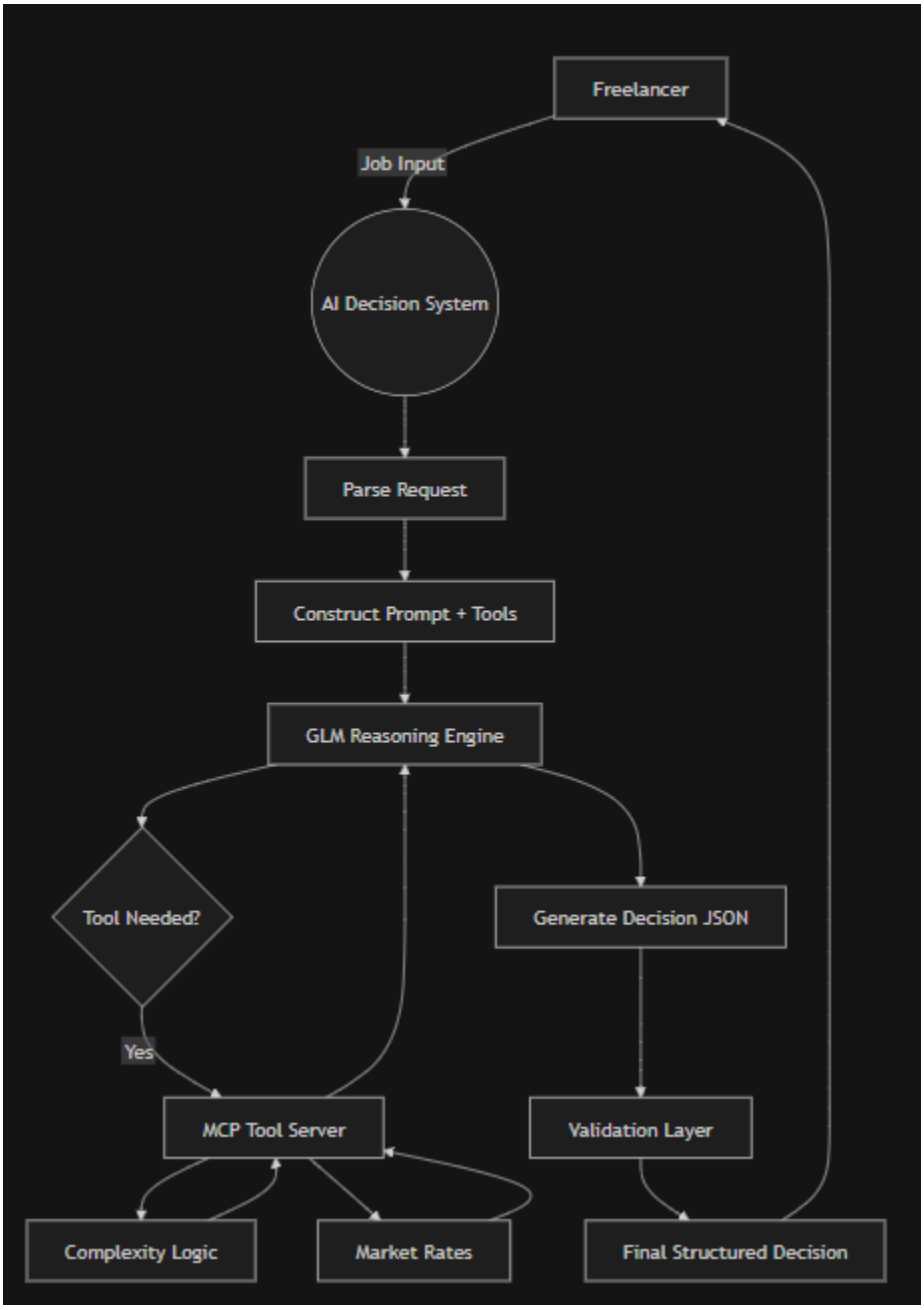
2.2.1. Frontend: Flutter - Frontend framework for cross-platform development

2.2.2. Backend: FastAPI – Python as the main language, RESTful API for the microservices, MCP for AI tooling

© UMHackathon 2026

2.3. Key Data Flows

2.3.1.1. Data Flow Diagram (DFD)



© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Functional Requirements & Scope

3.1. Minimum Viable Product:

#	Features	Description
1	Initial user prompt	User creates an initial prompt to get initial response
2	Prompt reply	Reply to the AI after its previous prompt
3	Creating new chat	Users can create a new chat

3.2. Non-Functional Requirements (NFRs)

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Scalability	The GLM must handle high amount of requests simultaneously	Proper queuing of requests and distributed hosting (to be implemented in the future)
Reliability	User prompts must be replied in a timely fashion.	Truncate user prompts to not be too long.
Maintainability	Frontend and Backend are deployed separately	Independent teams handling each department
Token Latency	The API calls to GLM must return a response within 1-3 minutes	Displaying user-facing loading state while awaiting for response
Cost Efficiency	Average costs of token per user session must not exceed a defined budget threshold.]	The length of the prompts should be fixed length. The inputs from the user are truncated or chunked before injecting

3.3. Out of Scope / Future Enhancements

Do identify the high-level features that the team planned for "Grand Vision" but not part of this UMHackathon 2026. [e.g. below]

a. *Caching of previous prompts for better contextualization*
b. *User login and registration*

© UMHackathon 2026

c. *Better input sanitization*
d. *Broad capability Freelance assistant*

4. Assumptions & Dependencies

4.1. Assumptions:

a. *Users have stable internet connections on their devices during the prompt*
b. *API provider has good outbound connection*

4.2. External Dependencies:

Tools	Purpose	Risks
Z Ai GLM API	The main decision-making AI and logic unit of the app.	High level - in case of GLM API rate limitation - core product will fail.
FastAPI	Endpoint creation for Frontend-Backend communication	High level - If the dependency goes down, Frontend cannot communicate with the Backend
Pydantic	User prompt sanitising and validation	Medium level - Invalid prompts may be sent to the GLM leading to confused responses

5. Project Management & Team Contributions

5.1. Project Timeline:

1. *Day 1 - 2: Planning. We wanted to make our app unique and stand out among competitors and avoid recreating an agentic workflow or basic AI chatting app.*
2. *Day 2 - 6: Frontend development. We wanted to get the UI working as early as possible for easier testing.*
3. *Day 6 - 10: Backend developmental. Due to the delay of API key release by z.ai, we could not work on the backend until Day 6.*

5.2. Team Members Role:

1. *Muhammad Yusuf bin Riduan - Team Leader and Frontend Lead. Worked on Frontend and Pitch Deck.*

© UMHackathon 2026

2. *Shawn Huang Qi Yang - Worked on Frontend, Backend and documentation.*
3. *Wan Wei Siang - Backend Lead. Worked on documentation.*
4. *Elsa Zara binti Fakhurazzi - Worked on Backend and documentation.*
5. *Loi Ming Han - Worked on Frontend and documentation.*

© UMHackathon 2026

Email: umhackathon@um.edu.my